Master Plan — Weather-Aware Truck Routing (Spotter.ai Assessment)

0. Constraints

- Time budget: 16 work hours / 4 days
- Stack: React + Django (required)
- Deliverables: GitHub repo + Hosted app + 5–10 min Loom walkthrough
- Grading weights: 40% Routes/Weather/Risk/Recommendation logic, 35% UI/Map/Heatmap smoothness, 10% Code quality, 5% Documentation

---

1. Governing Documents

Stored in `/docs` in the repo, treated as source of truth:

- `requirements.md` — SRS (what to build)
- `architecture.md` — high-level design (how components fit)
- `api-contract.md` — request/response interface spec
- `DECISIONS.md` — resolved ambiguities (see Section 5 below)

No further spec debate once coding starts — resolved decisions are final for this submission.

---

2. Architecture (layered, swappable providers)

API layer (Django views/serializers) ↓ Service layer (orchestration) ↓ Domain layer (pure functions — risk rules, recommendation) — zero I/O ↑ Provider adapters (routing, weather, geocoding) — behind interfaces

Rule: Domain layer never imports Django/requests/provider SDKs. Provider adapters never contain business rules. Swapping a provider or UI component should only touch one file/folder.

Frontend flow: `TripForm → useTripAnalysis hook → API call → MapView (routes + checkpoints + heatmap + slider) + RouteSummaryCard + WarningsBanner`

---

3. Repo & Deployment Strategy

- **One GitHub monorepo**, two folders: `backend/`, `frontend/`
- **Hosting:** Render for both backend (Web Service) and frontend (Static Site) — single dashboard, simpler env/CORS management
  - *(Alternative: Vercel for frontend if preferred — functionally equivalent)*
- **Cold-start mitigation (Render free tier):** Set up two free uptime pingers hitting `/api/health` every 5–10 min:
  - UptimeRobot (primary)
  - cron-job.org (redundant backup)
  - Keep both active through the entire review window (1–2 weeks post-submission)

- **Why not other platforms:** Koyeb (no longer reliably free), PythonAnywhere free tier (blocks outbound calls to non-whitelisted APIs — breaks routing/weather calls entirely), Railway (no longer free). Render + pinger is the most time-efficient, lowest-risk choice given the 16-hour budget.

Deployment config reference

Backend (Render Web Service)

- Root directory: `backend`
- Build command: `pip install -r requirements.txt`
- Start command: `gunicorn config.wsgi:application --bind 0.0.0.0:$PORT`
- Env vars: `SECRET_KEY`, `DEBUG=False`, `ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, routing/weather API keys

Frontend (Render Static Site or Vercel)

- Root directory: `frontend`
- Build command: `npm install && npm run build`
- Publish directory: `dist`
- Env var: `VITE_API_BASE_URL=https://<backend-url>`

---

4. Folder Structure

Backend backend/ config/ settings.py, urls.py, wsgi.py trip_analysis/ domain/ risk_rules.py, recommendation.py, types.py (pure logic, no I/O) providers/ routing/ (base.py, ors_provider.py) weather/ (base.py, open_meteo_provider.py) geocoding/ (base.py, ors_geocoder.py) services/ checkpoint_service.py ETA + checkpoint generation trip_analysis_service.py orchestration, builds contract response api/ serializers.py, views.py, urls.py, exceptions.py tests/ test_risk_rules.py test_recommendation.py test_checkpoint_service.py test_trip_analysis_service.py fixtures/ recorded sample provider responses healthcheck/ deploy-proof skeleton app (/api/health)

Frontend frontend/src/ types/trip.ts mirrors API contract exactly api/tripApi.ts single typed fetch function hooks/useTripAnalysis.ts loading/error/data state components/ TripForm/ MapView/ RouteLayer.tsx CheckpointMarkers.tsx HeatmapLayer.tsx ForecastSlider.tsx RouteSummaryCard/ WarningsBanner.tsx pages/HomePage.tsx App.tsx

---

5. Key Implementation Decisions (full detail in DECISIONS.md)

1. **Threshold boundaries:** half-open intervals, e.g. `moderate: 25 ≤ x < 35`
2. **Combining wind/rain/snow:** base risk = max severity of the three; load rules escalate on top (never reduce)
3. **Risk → miles conversion:** each checkpoint represents the segment ending at it; sum segment lengths by risk level; `average_risk_score` = distance-weighted mean of numeric severity (low=0 … no_travel=4)
4. **No Travel eligibility:** route with any `no_travel` checkpoint → `eligible: false`, excluded from ranking; if all ineligible → `recommended_route_id: null` + warning

5. ❤️ **routes / missing weather:** never fabricate; add explicit warnings; missing weather → `risk_level: "unknown"`, excluded from averages, never defaulted to Low
6. **Heatmap reference time:** backend server `now()` at request time; hourly steps `[0..48]`

---

6. Build Order (16 hrs / 4 days)

Phase	Hours	Focus
Day 0 (pre-work)	0.5h	GitHub repo init, skeleton deploy proof (backend health-check + frontend hello-world, live, CORS/env/build pipeline verified end-to-end)
Day 1	4h	Domain layer only: `risk_rules.py`, `recommendation.py`, full pytest boundary coverage. No Django/network calls yet.
Day 2	4h	Provider adapters (routing + geocoding + weather) + `checkpoint_service` (ETA interpolation) + `trip_analysis_service` orchestration, mocked tests
Day 3	4h	Django API endpoint wired to contract shape + validation + error handling; frontend form + API hook wired, confirm integration via raw JSON display
Day 4	4h	Map UI (routes, checkpoints, summary cards), heatmap + slider (precomputed, debounced), final deploy, uptime pinger setup, README assembly, Loom recording

---

7. API Contract Summary (reference)

`POST /api/routes/analyze`

- Request: `origin`, `destination`, `departure_time` (ISO 8601 w/ timezone), `load_weight_lb`, `checkpoint_interval_miles` (10/25/50)
- Response: `recommended_route_id`, `routes[]` (geometry, distance, duration, eligible, summary, checkpoints[]), `heatmap` (reference_time, available_forecast_hours, data), `warnings[]`
- Errors: `{ "error": { "code", "message" } }` — 400 (invalid input), 502 (provider failure), 503 (service unavailable)

---

8. Definition of Done (pre-submission checklist)

- ☐ All risk/recommendation boundary tests passing (pytest)
- ☐ API response matches contract exactly (field names, types, GeoJSON `[lng, lat]` order)
- ☐ 3 routes shown on map; recommended route visually distinct; ineligible routes flagged
- ☐ Heatmap slider smooth (precomputed data, no refetch per drag, debounced)
- ☐ No API keys exposed in frontend bundle
- ☐ Uptime pingers active, cold-start verified fixed (tested after 20+ min idle)
- ☐ README: setup steps, architecture diagram, decisions, how to run tests
- ☐ Loom (5–10 min): architecture → decisions → live demo (normal + edge case) → heatmap slider → tests passing → links

---

9. Current Status / Next Action

- [x] Governing docs written (requirements, architecture, api-contract, decisions)
- [x] Backend skeleton (Django health-check) working locally
- [x] Frontend skeleton (Vite hello-world) working locally, fetching backend successfully
- [ ] GitHub repo created & pushed
- [ ] Backend deployed to Render
- [ ] Frontend deployed to Render/Vercel
- [ ] CORS/env verified end-to-end on live URLs
- [ ] Uptime pingers configured
- [ ] → **Day 1: Domain layer (risk_rules.py, recommendation.py, pytest)**